Openi nCol ab

In [ ]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classificati
```

In [ ]:

```
data = pd.read_csv('/content/wireless_signal_quality_dataset.csv')
print("first five records")
print(data.head)
print("\nDataset Shape")
```

```
first five records
<bound method NDFrame.head of  Signal_Strength_dBm SNR_dB Interference_
dB Distance_m Frequency_GHz \
0  1                -41         6                   0          257                5.0
2  3                -86        25                  20          134                2.4
4  ..               -73        27                   0          385                2.4
95                  -57         5                   0          255                5.0
96                  -87        15                  24           61                5.0
97                  ...       ...                 ...          ...                ...
98                  -75        37                  11           17                2.4
99                  -88        11                  18          231                5.0
                    -57         6                  27          460                5.0
0  1                -49        25                  24          298                2.4
2  3                -46         8                  29          423                5.0
4  ..
95   Channel_Bandwidth_MHz Noise_Floor_dBm Connected_Devices \
96                     160             -100                24
97                     160              -96                41
98                      20              -72                13
99                     160              -86                 9
                        20              -79                32
                       ...              ...               ...
                       160              -80                34
                        20              -90                25
                       160              -76                24
                        20              -78                39
                        40              -71                16

     Estimated_Throughput_Mbps Quality
0  1                        18    Poor
2  3                        55    Poor
4  ..                       81    Fair
95                          15    Poor
96                          21    Poor
97                         ...     ...
98                         100    Fair
99                          15    Poor
                             1    Poor
                            51    Fair
                             1    Poor

[100 rows x 10 columns]>

Dataset Shape
```

In [ ]:

```
print("First five records:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nColumn Names:")
print(data.columns)

print("\nDataset Information:")
data.info()
```

```
First five records:
   Signal_Strength_dBm SNR_dB Interference_dB Distance_m Frequency_GHz
\
0                  -41      6               0         257           5.0
1                  -86     25              20         134           2.4
2                  -73     27               0         385           2.4
3                  -57      5               0         255           5.0
4                  -87     15              24          61           5.0

   Channel_Bandwidth_MHz Noise_Floor_dBm Connected_Devices \
0                    160            -100                24
1                    160             -96                41
2                     20             -72                13
3                    160             -86                 9
4                     20             -79                32

   Estimated_Throughput_Mbps Quality
0                         18    Poor
1                         55    Poor
2                         81    Fair
3                         15    Poor
4                         21    Poor

Dataset
Shape:     (100,
10)
Column Names:
Index(['Signal_Strength_dBm', 'SNR_dB', 'Interference_dB', 'Distance_m',
       'Frequency_GHz', 'Channel_Bandwidth_MHz', 'Noise_Floor_dBm',
       'Connected_Devices', 'Estimated_Throughput_Mbps', 'Quality'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Signal_Strength_dBm        100 non-null    int64
 1   SNR_dB                     100 non-null    int64
 2   Interference_dB            100 non-null    int64
 3   Distance_m                 100 non-null    int64
 4   Frequency_GHz              100 non-null    float64
 5   Channel_Bandwidth_MHz      100 non-null    int64
 6   Noise_Floor_dBm            100 non-null    int64
 7   Connected_Devices          100 non-null    int64
 8   Estimated_Throughput_Mbps  100 non-null    int64
 9   Quality                    100 non-null    object
dtypes: float64(1), int64(8), object(1)
```

```
memory usage: 7.9+ KB
```

In [ ]:
```
print("Missing Values:")
print(data.isnull().sum())
```

```
Missing Values:
Signal_Strength_dBm          0
SNR_dB                       0
Interference_dB              0
Distance_m                   0
Frequency_GHz                0
Channel_Bandwidth_MHz        0
Noise_Floor_dBm              0
Connected_Devices            0
Estimated_Throughput_Mbps    0
Quality                      0
dtype: int64
```

In [ ]:
```
print("Column Names:")
print(data.columns.tolist())

print("\nData Types:")
print(data.dtypes)
```

```
Column Names:
['Signal_Strength_dBm', 'SNR_dB', 'Interference_dB', 'Distance_m', 'Frequenc
y_GHz', 'Channel_Bandwidth_MHz', 'Noise_Floor_dBm', 'Connected_Devices', 'Es
timated_Throughput_Mbps', 'Quality']

Data Types:
Signal_Strength_dBm          int64
SNR_dB                       int64
Interference_dB              int64
Distance_m                   int64
Frequency_GHz              float64
Channel_Bandwidth_MHz        int64
Noise_Floor_dBm              int64
Connected_Devices            int64
Estimated_Throughput_Mbps    int64
Quality                     object
dtype: object
```

In [ ]:
```
print(data.columns.tolist())
```

```
['Signal_Strength_dBm', 'SNR_dB', 'Interference_dB', 'Distance_m', 'Frequenc
y_GHz', 'Channel_Bandwidth_MHz', 'Noise_Floor_dBm', 'Connected_Devices', 'Es
timated_Throughput_Mbps', 'Quality']
```

In [ ]:
```
X = data.drop('Quality', axis=1)
y = data['Quality']

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

```
Features:
   Signal_Strength_dBm SNR_dB Interference_dB Distance_m Frequency_GHz
\
0                  -41      6               0        257            5.0
```

```
1                -86         25                20          134           2.4
2                -73         27                 0          385           2.4
3                -57          5                 0          255           5.0
4                -87         15                24           61           5.0

   Channel_Bandwidth_MHz Noise_Floor_dBm Connected_Devices
0                    160            -100                24
1                    160             -96                41
2                     20             -72                13
3                    160             -86                 9
4                     20             -79                32

   Estimated_Throughput_Mbps
0                         18
1                         55
2                         81
3                         15
4                         21

Target:
0    Poor
1    Poor
2    Poor
3    Poor
4    Fair
Name: Quality, dtype: object
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,   test_size=0.2,
    random_state=42, stratify=y

)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (80, 9)
Testing data shape: (20, 9)
```

In [ ]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed successfully.")
```

```
Feature scaling completed successfully.
```

In [ ]:
```
svm_model = SVC(kernel='rbf', random_state=42)

svm_model.fit(X_train_scaled, y_train)

print("SVM model trained successfully!")
```

```
SVM model trained successfully!
```

In [ ]:
```
y_pred = svm_model.predict(X_test_scaled)

print("Predictions:")
```

```
p    t(y_ped[: 0])
```

Predictions:
['Poor' 'Poor' 'Poor' 'Poor' 'Fair' 'Fair' 'Poor' 'Poor' 'Poor' 'Poor'
 'Poor' 'Fair' 'Fair' 'Poor' 'Fair' 'Poor' 'Poor' 'Fair' 'Poor' 'Poor']

In [ ]:
```
accuracy = accuracy_score(y_test, y_pred)

print("SVM Accuracy:", accuracy)
print("SVM Accuracy (%):", accuracy * 100)
```

SVM Accuracy: 0.6
SVM Accuracy (%): 60.0

In [ ]:
```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

Confusion Matrix:
[[0 1 0 0]
[0 3 0 4]
[0 1 0 1]
[0 1 0 9]]

In [ ]:
```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
Excellent	0.00	0.00	0.00	1
Fair	0.50	0.43	0.46	7
Good	0.00	0.00	0.00	2
Poor	0.64	0.90	0.75	10
accuracy			0.60	20
macro avg	0.29	0.33	0.30	20
weighted avg	0.50	0.60	0.54	20

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))